Homework 5

Damion Huppert

In [46]:
```
using Pkg
Pkg.add("CSV")
using CSV
Pkg.add("DataFrames")
using DataFrames
Pkg.add("JuMP")
using JuMP
Pkg.add("HiGHS")
using HiGHS
Pkg.add("Plots")
using Plots
Pkg.add("Gurobi")
using Gurobi
Pkg.add("LinearAlgebra")
using LinearAlgebra
```

```
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
```

In [ ]:
```
# Read dataframe
df = CSV.read("nfl_scores-2022.csv", DataFrame)

# Get first 8 weeks
df_8 = filter(:week => x -> x <= 8, df)
println(df_8)
df_9 = filter(:week => x -> x == 9, df)
```

```julia
# Team mapping and home team margin
SCORE_DIFF = Dict{Tuple{String, String}, Int}()

for row in eachrow(df_8)
    if row.tm_location == "H"
        ht = row.tm_name
        at = row.opp_name
        ht_margin = row.tm_score - row.opp_score
    elseif row.tm_location == "A"
        ht = row.opp_name
        at = row.tm_name
        ht_margin = row.opp_score - row.tm_score
    end
    SCORE_DIFF[(ht, at)] = ht_margin
end

TEAMS = unique([ht for (ht,at) in keys(SCORE_DIFF)])
GAMES = keys(SCORE_DIFF)
GAMES
```

```
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.11/Project.toml`
  No Changes to `~/.julia/environments/v1.11/Manifest.toml`
```

123×7 DataFrame

Row	Column1	week	tm_name	opp_name	tm_location	tm_score	opp_score
	Int64	Int64	String15	String15	String1	Int64	Int64
1	0	1	Bills	Rams	A	31	10
2	1	1	Saints	Falcons	A	27	26
3	2	1	Browns	Panthers	A	26	24
4	3	1	Bears	49ers	H	19	10
5	4	1	Steelers	Bengals	A	23	20
6	5	1	Texans	Colts	H	20	20

7	6	1	Eagles	Lions	A	38
35						
8	7	1	Football Team	Jaguars	H	28
22						
9	8	1	Dolphins	Patriots	H	20
7						
10	9	1	Ravens	Jets	A	24
9						
11	10	1	Chiefs	Cardinals	A	44
21						
12	11	1	Vikings	Packers	H	23
7						
13	12	1	Giants	Titans	A	21
20						
14	13	1	Chargers	Raiders	H	24
19						
15	14	1	Buccaneers	Cowboys	A	19
3						
16	15	1	Seahawks	Broncos	H	17
16						
17	16	2	Chiefs	Chargers	H	27
24						
18	17	2	Giants	Panthers	H	19
16						
19	18	2	Jets	Browns	A	31
30						
20	19	2	Jaguars	Colts	H	24
0						
21	20	2	Lions	Football Team	H	36
27						
22	21	2	Dolphins	Ravens	A	42
38						
23	22	2	Buccaneers	Saints	A	20
10						
24	23	2	Patriots	Steelers	A	17
14						
25	24	2	Rams	Falcons	H	31
27						
26	25	2	49ers	Seahawks	H	27
7						
27	26	2	Cowboys	Bengals	H	20
17						
28	27	2	Cardinals	Raiders	A	29
23						
29	28	2	Broncos	Texans	H	16
9						
30	29	2	Packers	Bears	H	27
10						
31	30	2	Bills	Titans	H	41
7						
32	31	2	Eagles	Vikings	H	24
7						
33	32	3	Browns	Steelers	H	29

34	33	3	Dolphins	Bills	H	21
35	34	3	Panthers	Saints	H	22
36	35	3	Bears	Texans	H	23
37	36	3	Bengals	Jets	A	27
38	37	3	Colts	Chiefs	H	20
39	38	3	Vikings	Lions	H	28
40	39	3	Ravens	Patriots	A	37
41	40	3	Titans	Raiders	H	24
42	41	3	Eagles	Football Team	A	24
43	42	3	Jaguars	Chargers	A	38
44	43	3	Falcons	Seahawks	A	27
45	44	3	Rams	Cardinals	A	20
46	45	3	Packers	Buccaneers	A	14
47	46	3	Broncos	49ers	H	11
48	47	3	Cowboys	Giants	A	23
49	48	4	Bengals	Dolphins	H	27
50	49	4	Vikings	Saints	A	28
51	50	4	Falcons	Browns	H	23
52	51	4	Bills	Ravens	A	23
53	52	4	Giants	Bears	H	20
54	53	4	Titans	Colts	A	24
55	54	4	Cowboys	Football Team	H	25
56	55	4	Seahawks	Lions	A	48
57	56	4	Chargers	Texans	A	34
58	57	4	Eagles	Jaguars	H	29
59	58	4	Jets	Steelers	A	24

60	59	4	Cardinals	Panthers	A	26
16						
61	60	4	Raiders	Broncos	H	32
23						
62	61	4	Packers	Patriots	H	27
24						
63	62	4	Chiefs	Buccaneers	A	41
31						
64	63	4	49ers	Rams	H	24
9						
65	64	5	Colts	Broncos	A	12
9						
66	65	5	Giants	Packers	A	27
22						
67	66	5	Buccaneers	Falcons	H	21
15						
68	67	5	Bills	Steelers	H	38
3						
69	68	5	Vikings	Bears	H	29
22						
70	69	5	Chargers	Browns	A	30
28						
71	70	5	Patriots	Lions	H	29
0						
72	71	5	Texans	Jaguars	A	13
6						
73	72	5	Jets	Dolphins	H	40
17						
74	73	5	Saints	Seahawks	H	39
32						
75	74	5	Titans	Football Team	A	21
17						
76	75	5	49ers	Panthers	A	37
15						
77	76	5	Eagles	Cardinals	A	20
17						
78	77	5	Cowboys	Rams	A	22
10						
79	78	5	Ravens	Bengals	H	19
17						
80	79	5	Chiefs	Raiders	H	30
29						
81	80	6	Football Team	Bears	A	12
7						
82	81	6	Falcons	49ers	H	28
14						
83	82	6	Bengals	Saints	A	30
26						
84	83	6	Patriots	Browns	A	38
15						
85	84	6	Colts	Jaguars	H	34
27						
86	85	6	Jets	Packers	A	27

10						
87	86	6	Vikings	Dolphins	A	24
16						
88	87	6	Giants	Ravens	H	24
20						
89	88	6	Steelers	Buccaneers	H	20
18						
90	89	6	Rams	Panthers	H	24
10						
91	90	6	Seahawks	Cardinals	H	19
9						
92	91	6	Bills	Chiefs	A	24
20						
93	92	6	Eagles	Cowboys	H	26
17						
94	93	6	Chargers	Broncos	H	19
16						
95	94	7	Cardinals	Saints	H	42
34						
96	95	7	Bengals	Falcons	H	35
17						
97	96	7	Panthers	Buccaneers	H	21
3						
98	97	7	Ravens	Browns	H	23
20						
99	98	7	Titans	Colts	H	19
10						
100	99	7	Cowboys	Lions	H	24
6						
101	100	7	Football Team	Packers	H	23
21						
102	101	7	Giants	Jaguars	A	23
17						
103	102	7	Raiders	Texans	H	38
20						
104	103	7	Jets	Broncos	A	16
9						
105	104	7	Chiefs	49ers	A	44
23						
106	105	7	Seahawks	Chargers	A	37
23						
107	106	7	Dolphins	Steelers	H	16
10						
108	107	7	Bears	Patriots	A	33
14						
109	108	8	Ravens	Buccaneers	A	27
22						
110	109	8	Broncos	Jaguars	A	21
17						
111	110	8	Dolphins	Lions	A	31
27						
112	111	8	Eagles	Steelers	H	35
13						

113	112	8	Falcons	Panthers	H	37
34						
114	113	8	Cowboys	Bears	H	49
29						
115	114	8	Vikings	Cardinals	H	34
26						
116	115	8	Saints	Raiders	H	24
0						
117	116	8	Patriots	Jets	A	22
17						
118	117	8	Titans	Texans	A	17
10						
119	118	8	49ers	Rams	A	31
14						
120	119	8	Football Team	Colts	A	17
16						
121	120	8	Seahawks	Giants	H	27
13						
122	121	8	Bills	Packers	H	27
17						
123	122	8	Browns	Bengals	H	32
13						

Out[ ]:
```
KeySet for a Dict{Tuple{String, String}, Int64} with 123 entries. Keys:
  ("Steelers", "Buccaneers")
  ("Saints", "Raiders")
  ("Cowboys", "Football Team")
  ("Browns", "Jets")
  ("Football Team", "Titans")
  ("Jets", "Dolphins")
  ("Dolphins", "Steelers")
  ("Buccaneers", "Packers")
  ("Falcons", "49ers")
  ("Ravens", "Browns")
  ("Vikings", "Lions")
  ("Chargers", "Seahawks")
  ("Seahawks", "Giants")
  ("Falcons", "Browns")
  ("49ers", "Seahawks")
  ("Cardinals", "Chiefs")
  ("Saints", "Seahawks")
  ("Eagles", "Steelers")
  ("Panthers", "Browns")
  ("Titans", "Colts")
  ("Lions", "Dolphins")
  ("Broncos", "Texans")
  ("Football Team", "Packers")
  ("Texans", "Titans")
  ("Raiders", "Cardinals")
  ⋮
```

Question 1-1

Decision Variables:

$$x_i \quad \text{Perfomance of team i}$$

$$\beta \quad \text{Homefield advantage parameter}$$

Constraints

$$\sum_{i=1}^{32} x_i = 0 \quad \text{Rating of all teams combined sums to 0}$$

Objective

$$\min \sum_{i,j \in G} (x_i - x_j + \beta - \Delta_{ij})^2 \quad \text{Minimize the squared error for all games, i is the ho}$$

Question 1-2

In [7]:
```julia
# Map team names to indices
team_idx = Dict(t => i for (i, t) in enumerate(TEAMS))

# Build model
model = Model(HiGHS.Optimizer)
@variable(model, x[1:length(TEAMS)])   # team ratings
@variable(model, β)                    # home field advantage

@constraint(model, sum(x) == 0)

@objective(model, Min, sum(
    (x[team_idx[ht]] - x[team_idx[at]] + β - SCORE_DIFF[(ht, at)])^2
    for (ht, at) in GAMES
))

set_silent(model)  # suppress output
optimize!(model)

# Print team ratings
ratings = [(t, value(x[team_idx[t]])) for t in TEAMS]
sort!(ratings, by = x -> -x[2])  # sort descending
println("Team Ratings (highest to lowest):")
for (team, rating) in ratings
    println(rpad(team, 20), round(rating, digits=3))
end

println("\nHome Field Advantage: β = ", round(value(β), digits=3))
```

```
Team Ratings (highest to lowest):
Bills               15.777
Eagles              9.666
Chiefs              7.811
Ravens              6.375
Cowboys             5.653
Bengals             4.268
```

Vikings	3.573
Jets	2.535
49ers	2.511
Patriots	1.683
Dolphins	1.273
Giants	0.705
Buccaneers	0.23
Browns	-0.475
Seahawks	-0.618
Jaguars	-0.995
Falcons	-1.092
Saints	-1.158
Packers	-1.285
Titans	-2.057
Cardinals	-2.593
Bears	-2.742
Raiders	-3.531
Broncos	-3.784
Rams	-3.805
Football Team	-4.042
Panthers	-4.686
Steelers	-4.779
Lions	-5.245
Chargers	-5.292
Colts	-5.333
Texans	-8.547

Home Field Advantage: β = 1.906

Bills	15.777
Eagles	9.666
Chiefs	7.811
Ravens	6.375
Cowboys	5.653
Bengals	4.268
Vikings	3.573
Jets	2.535
49ers	2.511
Patriots	1.683
Dolphins	1.273
Giants	0.705
Buccaneers	0.23
Browns	-0.475
Seahawks	-0.618
Jaguars	-0.995
Falcons	-1.092
Saints	-1.158
Packers	-1.285
Titans	-2.057
Cardinals	-2.593
Bears	-2.742
Raiders	-3.531
Broncos	-3.784
Rams	-3.805

```
Football Team       -4.042
Panthers            -4.686
Steelers            -4.779
Lions               -5.245
Chargers            -5.292
Colts               -5.333
Texans              -8.547

Home Field Advantage: β = 1.906
```

Question 1-3

Bengals ranking: 4.268

Chiefs rankning: 7.811

Homefield advantage: 1.906

$$4.268 - 7.811 + 1.906 = -1.637$$

Cheifs win 🥰

Question 1-4

In [13]:

```julia
# Map team names to indices
team_idx = Dict(t => i for (i, t) in enumerate(TEAMS))

# Build model
model = Model(HiGHS.Optimizer)
@variable(model, x[1:length(TEAMS)])      # team ratings
@variable(model, β)                       # home field advantage
@variable(model, t[1:length(GAMES)] ≥ 0)  # absolute value terms

@constraint(model, sum(x) == 0)
# Add abs-value constraints per game
for (g_idx, (ht, at)) in enumerate(GAMES)
  i = team_idx[ht]
  j = team_idx[at]
  Δ = SCORE_DIFF[(ht, at)]
  @constraint(model, x[i] - x[j] + β - Δ ≤ t[g_idx])
  @constraint(model, -(x[i] - x[j] + β - Δ) ≤ t[g_idx])
end

@objective(model, Min, sum(t))

set_silent(model)  # suppress output
optimize!(model)

# Print team ratings
ratings = [(t, value(x[team_idx[t]])) for t in TEAMS]
sort!(ratings, by = x -> -x[2])  # sort descending
println("Team Ratings (highest to lowest):")
for (team, rating) in ratings
    println(rpad(team, 20), round(rating, digits=3))
```

```
end

println("\nHome Field Advantage: β = ", round(value(β), digits=3))
```

```
Team Ratings (highest to lowest):
Bills               16.367
Eagles              14.367
Chiefs              10.617
Cowboys             8.867
Ravens              7.867
Bengals             7.617
Cardinals           3.617
Vikings             2.117
Titans              1.617
Raiders             1.367
Chargers            0.867
Giants              0.117
Packers             0.117
Jets                -0.133
Buccaneers          -1.133
Patriots            -1.133
49ers               -1.133
Dolphins            -1.633
Broncos             -1.883
Seahawks            -2.633
Saints              -2.633
Browns              -2.883
Football Team       -3.383
Rams                -4.883
Falcons             -5.383
Colts               -5.633
Steelers            -5.883
Bears               -6.133
Panthers            -6.633
Lions               -7.383
Texans              -7.383
Jaguars             -7.633

Home Field Advantage: β = 1.75
```

Question 1-5

In [17]:
```
df_9 = filter(:week => x -> x == 9, df)

correct = 0
total = 0

for row in eachrow(df_9)
    if row.tm_location == "H"
        ht = row.tm_name
        at = row.opp_name
        predicted_margin = value(x[team_idx[ht]]) - value(x[team_idx[at]]) +
        actual_margin = row.tm_score - row.opp_score
    else
```

```
        ht = row.opp_name
        at = row.tm_name
        predicted_margin = value(x[team_idx[ht]]) - value(x[team_idx[at]]) +
        actual_margin = row.opp_score - row.tm_score
    end

    predicted_winner = predicted_margin > 0 ? ht : at
    actual_winner = actual_margin > 0 ? ht : at
    is_correct = predicted_winner == actual_winner

    correct += is_correct ? 1 : 0
    total += 1

    println(ht, " vs ", at,
        " | Predicted Margin: ", round(predicted_margin, digits=2),
        " | Actual Margin: ", round(actual_margin, digits=2),
        " | Predicted Winner: ", predicted_winner,
        " | Actual Winner: ", actual_winner,
        " | Correct: ", is_correct)
end

println("\nCorrect Week 9 Predictions: ", correct, "/", total)
```

```
Texans vs Eagles | Predicted Margin: -20.0 | Actual Margin: -12.0 | Predicte
d Winner: Eagles | Actual Winner: Eagles | Correct: true
Jets vs Bills | Predicted Margin: -14.75 | Actual Margin: 3.0 | Predicted Wi
nner: Bills | Actual Winner: Jets | Correct: false
Lions vs Packers | Predicted Margin: -5.75 | Actual Margin: 6.0 | Predicted
Winner: Packers | Actual Winner: Lions | Correct: false
Football Team vs Vikings | Predicted Margin: -3.75 | Actual Margin: -3.0 | P
redicted Winner: Vikings | Actual Winner: Vikings | Correct: true
Falcons vs Chargers | Predicted Margin: -4.5 | Actual Margin: -3.0 | Predict
ed Winner: Chargers | Actual Winner: Chargers | Correct: true
Bengals vs Panthers | Predicted Margin: 16.0 | Actual Margin: 21.0 | Predict
ed Winner: Bengals | Actual Winner: Bengals | Correct: true
Bears vs Dolphins | Predicted Margin: -2.75 | Actual Margin: -3.0 | Predicte
d Winner: Dolphins | Actual Winner: Dolphins | Correct: true
Patriots vs Colts | Predicted Margin: 6.25 | Actual Margin: 23.0 | Predicted
Winner: Patriots | Actual Winner: Patriots | Correct: true
Jaguars vs Raiders | Predicted Margin: -7.25 | Actual Margin: 7.0 | Predicte
d Winner: Raiders | Actual Winner: Jaguars | Correct: false
Cardinals vs Seahawks | Predicted Margin: 8.0 | Actual Margin: -10.0 | Predi
cted Winner: Cardinals | Actual Winner: Seahawks | Correct: false
Buccaneers vs Rams | Predicted Margin: 5.5 | Actual Margin: 3.0 | Predicted
Winner: Buccaneers | Actual Winner: Buccaneers | Correct: true
Chiefs vs Titans | Predicted Margin: 10.75 | Actual Margin: 3.0 | Predicted
Winner: Chiefs | Actual Winner: Chiefs | Correct: true
Saints vs Ravens | Predicted Margin: -8.75 | Actual Margin: -14.0 | Predicte
d Winner: Ravens | Actual Winner: Ravens | Correct: true

Correct Week 9 Predictions: 9/13
Eagles | Predicted Margin: -20.0 | Actual Margin: -12.0 | Predicted Winner:
Eagles | Actual Winner: Eagles | Correct: true
Jets vs Bills | Predicted Margin: -14.75 | Actual Margin: 3.0 | Predicted Wi
```

```
nner: Bills | Actual Winner: Jets | Correct: false
Lions vs Packers | Predicted Margin: -5.75 | Actual Margin: 6.0 | Predicted
Winner: Packers | Actual Winner: Lions | Correct: false
Football Team vs Vikings | Predicted Margin: -3.75 | Actual Margin: -3.0 | P
redicted Winner: Vikings | Actual Winner: Vikings | Correct: true
Falcons vs Chargers | Predicted Margin: -4.5 | Actual Margin: -3.0 | Predict
ed Winner: Chargers | Actual Winner: Chargers | Correct: true
Bengals vs Panthers | Predicted Margin: 16.0 | Actual Margin: 21.0 | Predict
ed Winner: Bengals | Actual Winner: Bengals | Correct: true
Bears vs Dolphins | Predicted Margin: -2.75 | Actual Margin: -3.0 | Predicte
d Winner: Dolphins | Actual Winner: Dolphins | Correct: true
Patriots vs Colts | Predicted Margin: 6.25 | Actual Margin: 23.0 | Predicted
Winner: Patriots | Actual Winner: Patriots | Correct: true
Jaguars vs Raiders | Predicted Margin: -7.25 | Actual Margin: 7.0 | Predicte
d Winner: Raiders | Actual Winner: Jaguars | Correct: false
Cardinals vs Seahawks | Predicted Margin: 8.0 | Actual Margin: -10.0 | Predi
cted Winner: Cardinals | Actual Winner: Seahawks | Correct: false
Buccaneers vs Rams | Predicted Margin: 5.5 | Actual Margin: 3.0 | Predicted
Winner: Buccaneers | Actual Winner: Buccaneers | Correct: true
Chiefs vs Titans | Predicted Margin: 10.75 | Actual Margin: 3.0 | Predicted
Winner: Chiefs | Actual Winner: Chiefs | Correct: true
Saints vs Ravens | Predicted Margin: -8.75 | Actual Margin: -14.0 | Predicte
d Winner: Ravens | Actual Winner: Ravens | Correct: true

Correct Week 9 Predictions: 9/13
```

Question 2-1

In [19]:

```julia
using JuMP, HiGHS, Printf

k = 60              # number of seconds
dt = 1 / 3600  # seconds to hours

m = Model(HiGHS.Optimizer)
set_silent(m)
# ANAKIN
@variable(m, xa[1:2,1:k])  # position at each time (miles)
@variable(m, va[1:2,1:k])  # velocity at each time (mph)
@variable(m, ua[1:2,1:k])  # thruster input at each time
# PALPATINE
@variable(m, xp[1:2,1:k])  # position at each time (miles)
@variable(m, vp[1:2,1:k])  # velocity at each time (mph)
@variable(m, up[1:2,1:k])  # thruster input at each time

# satisfy the dynamics (with zero initial velocity)
# Note the use of '.' form of constraints, to make the constraints hold for

@constraint(m, va[:,1] .== [0.0;20.0]) # initial speed of ANAKIN (20 mph nor
@constraint(m, xa[:,1] .== [0;0.0]) # initial position of ANAKIN ((0,0) he i
@constraint(m, vp[:,1] .== [30.0;0.0]) # initial speed of PALPATINE (30 mph
@constraint(m, xp[:,1] .== [0.5;0.0]) # initial position of PALPATINE ((1/2,

# Dynamics constraints
```

```
for t in 1:k-1
  @constraint(m, xa[:,t+1] .== xa[:,t] + dt * va[:,t])
  @constraint(m, va[:,t+1] .== va[:,t] + ua[:,t])

  @constraint(m, xp[:,t+1] .== xp[:,t] + dt * vp[:,t])
  @constraint(m, vp[:,t+1] .== vp[:,t] + up[:,t])
end

@constraint(m, xa[:,k] .== xp[:,k])

@objective(m, Min, sum(ua.^2) + sum(up.^2))
optimize!(m)

# Results
final_pos = value.(xa[:,k])
total_energy = objective_value(m)
@printf("Final position: (%.4f miles, %.4f miles)\n", final_pos...)
@printf("Minimum total energy: %.4f\n", total_energy)
```

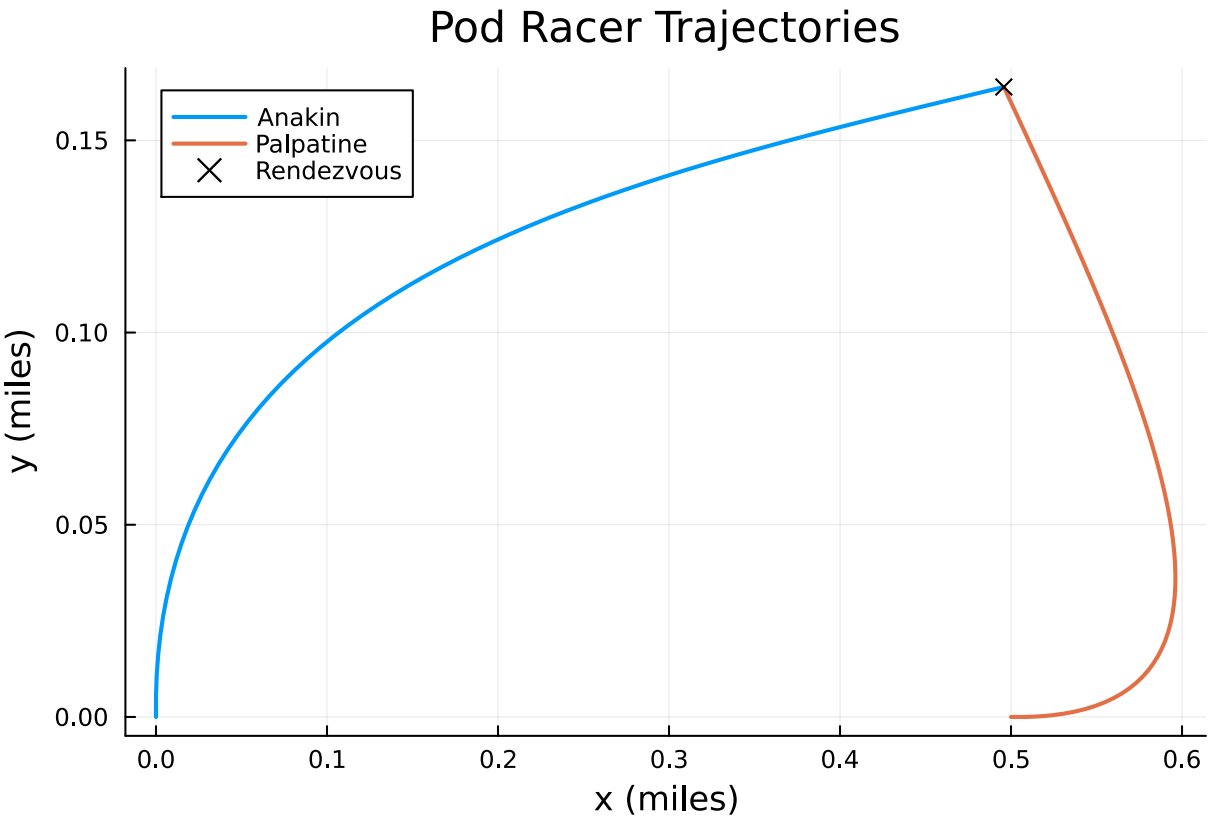
```
Final position: (0.4958 miles, 0.1639 miles)
Minimum total energy: 105.9307
```

Question 2-2

In [20]:

```
xa_vals = value.(xa)
xp_vals = value.(xp)

plot(xa_vals[1,:], xa_vals[2,:], label="Anakin", lw=2)
plot!(xp_vals[1,:], xp_vals[2,:], label="Palpatine", lw=2)
scatter!([final_pos[1]], [final_pos[2]], label="Rendezvous", color=:black, m
xlabel!("x (miles)")
ylabel!("y (miles)")
title!("Pod Racer Trajectories")
```

Out[20]:



Question 2-3

In [21]:

```
using JuMP, HiGHS, Printf

k = 60              # number of seconds
dt = 1 / 3600  # seconds to hours

m = Model(HiGHS.Optimizer)
set_silent(m)
# ANAKIN
@variable(m, xa[1:2,1:k])  # position at each time (miles)
@variable(m, va[1:2,1:k])  # velocity at each time (mph)
@variable(m, ua[1:2,1:k])  # thruster input at each time
# PALPATINE
@variable(m, xp[1:2,1:k])  # position at each time (miles)
@variable(m, vp[1:2,1:k])  # velocity at each time (mph)
@variable(m, up[1:2,1:k])  # thruster input at each time

# satisfy the dynamics (with zero initial velocity)
# Note the use of '.' form of constraints, to make the constraints hold for

@constraint(m, va[:,1] .== [0.0;20.0]) # initial speed of ANAKIN (20 mph nor
@constraint(m, va[:,60] .== [0.0;0.0]) # final speed of ANAKIN (0 mph)
@constraint(m, xa[:,1] .== [0;0.0]) # initial position of ANAKIN ((0,0) he i
@constraint(m, vp[:,1] .== [30.0;0.0]) # initial speed of PALPATINE (30 mph
@constraint(m, vp[:,60] .== [0.0;0.0]) # final speed of PALPATINE (0 mph)
@constraint(m, xp[:,1] .== [0.5;0.0]) # initial position of PALPATINE ((1/2,

# Dynamics constraints
```

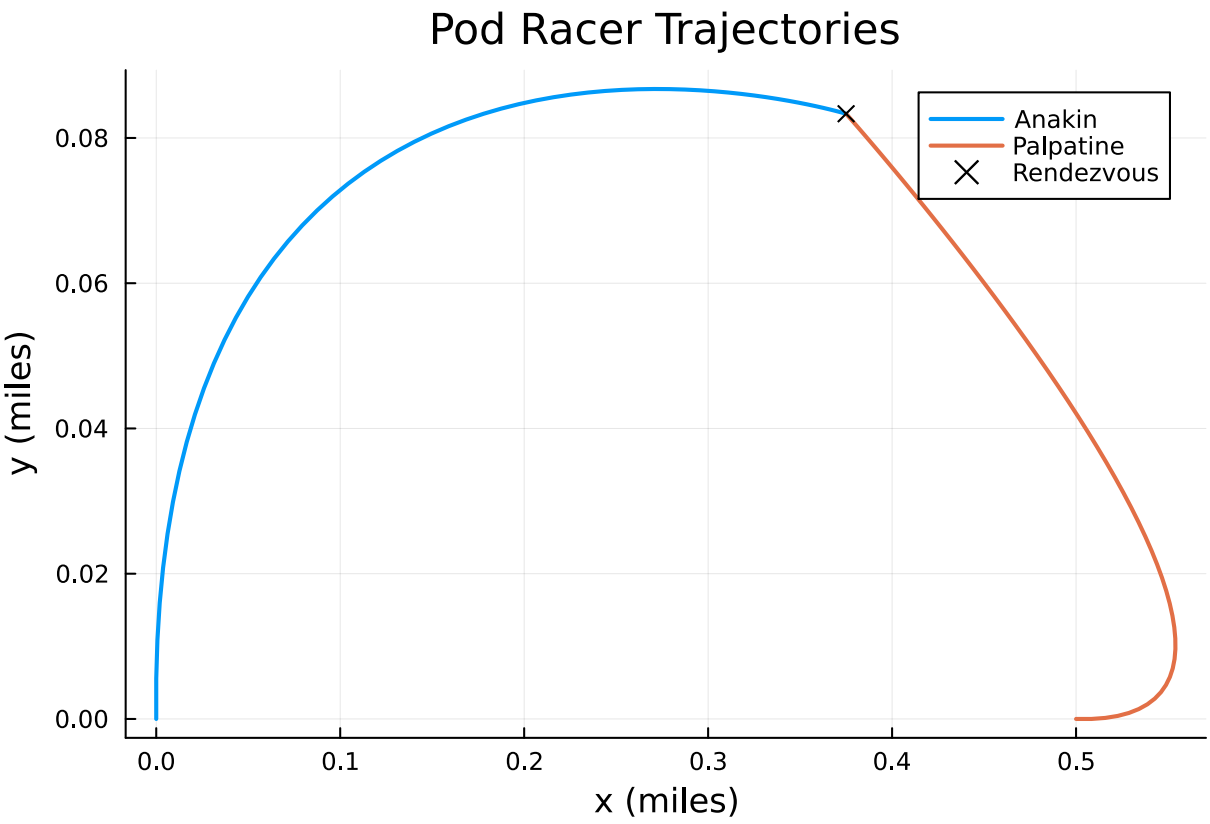
```
for t in 1:k-1
  @constraint(m, xa[:,t+1] .== xa[:,t] + dt * va[:,t])
  @constraint(m, va[:,t+1] .== va[:,t] + ua[:,t])

  @constraint(m, xp[:,t+1] .== xp[:,t] + dt * vp[:,t])
  @constraint(m, vp[:,t+1] .== vp[:,t] + up[:,t])
end

@constraint(m, xa[:,k] .== xp[:,k])

@objective(m, Min, sum(ua.^2) + sum(up.^2))
optimize!(m)

# Results
final_pos = value.(xa[:,k])
total_energy = objective_value(m)
@printf("Final position: (%.4f miles, %.4f miles)\n", final_pos...)
@printf("Minimum total energy: %.4f\n", total_energy)
```

```
Final position: (0.3750 miles, 0.0833 miles)
Minimum total energy: 245.5874
```

In [22]:

```
xa_vals = value.(xa)
xp_vals = value.(xp)

plot(xa_vals[1,:], xa_vals[2,:], label="Anakin", lw=2)
plot!(xp_vals[1,:], xp_vals[2,:], label="Palpatine", lw=2)
scatter!([final_pos[1]], [final_pos[2]], label="Rendezvous", color=:black, m
xlabel!("x (miles)")
ylabel!("y (miles)")
title!("Pod Racer Trajectories")
```

Out[22]:



Question 3-1

Decision Variables

$$h_i \quad \text{CURRENT percent holding of asset i}$$

$$x_i \quad \text{NEW portfolio holding of asset i}$$

$$\text{buy}_i \quad \text{Amount of asset i to buy}$$

$$\text{sell}_i \quad \text{Amount of asset i to sell}$$

Objective

$$\min \sum_{i \in A} (\text{sell}_i + \text{buy}_i) \quad \text{Minimize the total number of dollars that must be bought c}$$

Subject To

$$x_i = h_i + \text{buy}_i - \text{sell}_i$$

$$\sqrt{(x-b)^T Q (x-b)} \leq 0.10$$

$$\sum_{i \in A} x_i = 1$$

In [25]:
```
df = CSV.read("folio_mean.csv", DataFrame, header=false, delim=',')
```

```
df2 = CSV.read("folio_cov.csv",DataFrame,header=false,delim=',')
df3 = CSV.read("folio_holding_benchmark.csv", DataFrame, header=false, delim

(n,mmm) = size(df)
mu = -100/7*365*Vector(df[1:n,1])

Q = 0.5* (100/7*365)^2 * Matrix(df2)
h = Vector(df3[1:n,1])
b = Vector(df3[1:n,2])
benchmark_return = mu'b
holdings_return = mu'*h
active_risk = sqrt((h-b)'Q*(h-b))
println("Benchmark expected return: ", benchmark_return)
println("Holdings expected return: ", holdings_return)
println("Active Risk: ", active_risk)
```

```
Benchmark expected return: 9.48798378650461
Holdings expected return: 5.67073489826738
Active Risk: 32.90926641232401

Holdings expected return: 5.67073489826738
Active Risk: 32.90926641232401
```

In [43]:

```
model = Model(Gurobi.Optimizer)

@variable(model, x[1:n] >= 0)
@variable(model, buy[1:n] >= 0)
@variable(model, sell[1:n] >= 0)

@objective(model, Min, sum(buy + sell))

@constraint(model, x .== h .+ buy .- sell)
@constraint(model, sum(x) == 1)
@constraint(model, (x-b)'*Q*(x-b) <= 0.01)

set_silent(model)
optimize!(model)

if termination_status(model) == MOI.OPTIMAL
  obj_value = objective_value(model)
  x_opt = value.(x)
  buy_opt = value.(buy)
  sell_opt = value.(sell)

  println("Total number of dollars transacted: ", obj_value*1000000000)
  println("Expected return: ", value(mu'x))
  println("Active risk: ", value(sqrt((x-b)'Q*(x-b))))
else
  println("Optimization did not succeed. Status: ", termination_status(model
end
```

```
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Total number of dollars transacted: 1.8965096546548574e9
Expected return: 9.484021719438463
Active risk: 0.09993267508857184
```

Question 3-3

In [47]:

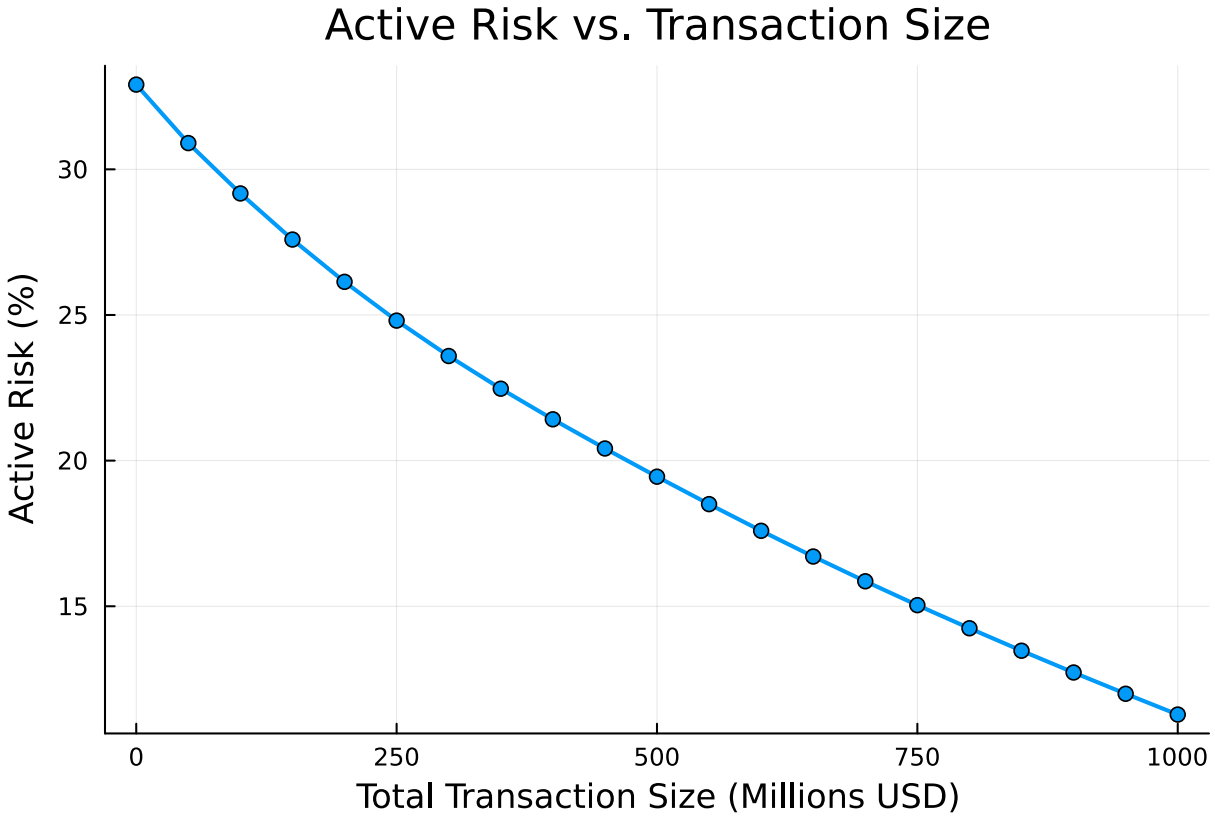
```
transaction_limits = 0:50e6:1e9
active_risks = Float64[]
expected_returns = Float64[]

# Loop over different transaction limits
for T in transaction_limits
    model = Model(Gurobi.Optimizer)
    set_silent(model)

    @variable(model, x[1:n] >= 0)
    @variable(model, buy[1:n] >= 0)
    @variable(model, sell[1:n] >= 0)

    @constraint(model, x .== h .+ buy .- sell)
    @constraint(model, sum(x) == 1)
    @constraint(model, sum(buy) + sum(sell) <= T / 1e9)  # Divide by 1B to m

    # Minimize active risk
    @objective(model, Min, (x - b)' * Q * (x - b))

    optimize!(model)

    if termination_status(model) == MOI.OPTIMAL
        push!(active_risks, sqrt(objective_value(model)))
        push!(expected_returns, dot(mu, value.(x)))
    else
        push!(active_risks, NaN)
        push!(expected_returns, NaN)
        println("Solver failed at transaction limit \$", T)
    end
end

# Plot active risk vs transaction size
plot(transaction_limits ./ 1e6, active_risks,
    xlabel="Total Transaction Size (Millions USD)",
    ylabel="Active Risk (%)",
    title="Active Risk vs. Transaction Size",
    legend=false,
    lw=2,
    marker=:circle)
```

```
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
```

```
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
```

```
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
```

Out[47]:



Question 4-1

Decision Variables:

$$x_i \quad 1 \text{ if the factory opens } 0 \text{ otherwise}$$

$$b_{ib} \quad \text{cupcakes shipped from plant i to bakery b}$$

Parameters:

- s_i: Maximum supply capacity of plant i
- c_i: Fixed cost of opening plant i
- t_{ib}: Transportation cost per cupcake from plant i to bakery b
- d_b: Monthly demand of bakery b

Objective:

$$\min \sum_{i \in P} c_i x_i + \sum_{i \in P} \sum_{b \in B} t_{ib} \cdot b_{ib}$$

Subject to:

$$\sum_{i \in P} b_{ib} = d_b \quad \forall b \in B$$

$$\sum_{b \in B} b_{ib} \leq s_i \cdot x_i \quad \forall i \in P$$

$$b_{ib} \geq 0 \quad \forall i \in P, b \in B$$

$$x_i \in \{0, 1\} \quad \forall i \in P$$

Question 4-2

In [48]:

```
plant_supply = [1600, 1500, 1200]
bakery_demand = [1600, 900, 1000]
fixed_cost = [10000, 11000, 9000]
cost = [
    15 10 12;
    20 18 21;
    17 15 11
]

num_plants = 3
num_bakeries = 3

model = Model(Gurobi.Optimizer)
set_silent(model)

@variable(model, x[1:num_plants, 1:num_bakeries] >= 0)
@variable(model, y[1:num_plants], Bin)

@objective(model, Min, sum(cost[i,j]*x[i,j] for i in 1:3, j in 1:3) + sum(fi

for j in 1:num_bakeries
    @constraint(model, sum(x[i,j] for i in 1:num_plants) == bakery_demand[j]
end

for i in 1:num_plants
    @constraint(model, sum(x[i,j] for j in 1:num_bakeries) <= plant_supply[i
end

optimize!(model)

if termination_status(model) == MOI.OPTIMAL
    println("✅ Minimum Monthly Cost: \$", objective_value(model))
    println("📦 Open Plants:")
    for i in 1:num_plants
        println("  Plant $i: ", value(y[i]) > 0.5 ? "Open" : "Closed")
    end
    println("\n🚚 Shipping Plan:")
    for i in 1:num_plants
        for j in 1:num_bakeries
            shipped = value(x[i,j])
```

```
            if shipped > 1e-6
                println("  Ship ", round(shipped), " cupcakes from Plant $i
            end
        end
    end
else
    println("Optimization failed with status: ", termination_status(model))
end
```

```
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
✅ Minimum Monthly Cost: $77900.0
📦 Open Plants:
  Plant 1: Open
  Plant 2: Open
  Plant 3: Open

🚚 Shipping Plan:
  Ship 700.0 cupcakes from Plant 1 to Bakery 1
  Ship 900.0 cupcakes from Plant 1 to Bakery 2
  Ship 700.0 cupcakes from Plant 2 to Bakery 1
  Ship 200.0 cupcakes from Plant 3 to Bakery 1
  Ship 1000.0 cupcakes from Plant 3 to Bakery 3
```

Question 5-1

Sets:

- $B = \{1, 2, 3\}$: Boilers
- $T = \{1, 2, 3\}$: Turbines

Decision Variables:

$$x_i \quad \text{tons of steam produced by boiler } i$$

$$b_i \in \{0, 1\} \quad 1 \text{ if boiler i is used, 0 otherwise}$$

$$s_j \quad \text{tons of steam processed by turbine } j$$

$$z_j \in \{0, 1\} \quad 1 \text{ if turbine } j \text{ is used, 0 otherwise}$$

Parameters:

$$c_i \quad \text{cost per ton produced from boiler i}$$

$$q_j \quad \text{cost per ton proccessed from turbine j}$$

$$p_j \quad \text{power produced per ton of steam}$$

Objective:

$$\min \sum_{i=1}^{3} c_i x_i + \sum_{j=1}^{3} q_j s_j$$

Subject to:

$$\min_i b_i \leq x_i \leq \max_i b_i \quad \forall i \in B$$

$$\min_j z_j \leq s_j \leq \max_j z_j \quad \forall j \in T$$

$$\sum_{i=1}^{3} x_i = \sum_{j=1}^{3} s_j$$

$$\sum_{j=1}^{3} p_j s_j \geq 8000$$

$$b_i, z_j \in \{0, 1\}$$

Question 5-2

In [ ]:
```
boiler_min = [400, 200, 300]
boiler_max = [1000, 900, 800]
boiler_cost = [10, 8, 7]
turbine_min = [300, 500, 600]
turbine_max = [600, 800, 900]
turbine_power = [4, 5, 6]
turbine_cost = [2, 3, 4]
required_power = 8000

model = Model(Gurobi.Optimizer)
set_silent(model)

@variable(model, 0 <= x[1:3])
@variable(model, b[1:3], Bin)
@variable(model, 0 <= s[1:3])
@variable(model, z[1:3], Bin)

@objective(model, Min, sum(boiler_cost[i] * x[i] for i in 1:3) + sum(turbine

for i in 1:3
    @constraint(model, boiler_min[i] * b[i] <= x[i])
    @constraint(model, x[i] <= boiler_max[i] * b[i])
end

for j in 1:3
    @constraint(model, turbine_min[j] * z[j] <= s[j])
    @constraint(model, s[j] <= turbine_max[j] * z[j])
end

@constraint(model, sum(x) == sum(s))
```

```
@constraint(model, sum(turbine_power[j] * s[j] for j in 1:3) >= required_pow

optimize!(model)

if termination_status(model) == MOI.OPTIMAL
    println("✅ Minimum Total Cost: \$", objective_value(model))
    for i in 1:3
        println("Boiler B$i: ", value(y[i]) > 0.5 ? "On, " : "Off, ",
                "Steam Produced = ", round(value(x[i]), digits=2), " tons")
    end
    for j in 1:3
        println("Turbine T$j: ", value(z[j]) > 0.5 ? "On, " : "Off, ",
                "Steam Processed = ", round(value(s[j]), digits=2), " tons,
                "Power = ", round(turbine_power[j] * value(s[j]), digits=2),
    end
else
    println("❌ Optimization failed: ", termination_status(model))
end
```

```
Set parameter Username
Set parameter LicenseID to value 2649580
Academic license - for non-commercial use only - expires 2026-04-09
✅ Minimum Total Cost: $15720.0
Boiler B1: On, Steam Produced = 0.0 tons
Boiler B2: On, Steam Produced = 620.0 tons
Boiler B3: On, Steam Produced = 800.0 tons
Turbine T1: Off, Steam Processed = 0.0 tons, Power = 0.0 kWh
Turbine T2: On, Steam Processed = 520.0 tons, Power = 2600.0 kWh
Turbine T3: On, Steam Processed = 900.0 tons, Power = 5400.0 kWh
```